

PasswordStore Audit Report

Version 1.0

Mohammad Ayaan
moayaan.com

November 8, 2026

PasswordStore Audit Report

Mohammad Ayaan

November 8, 2026

Prepared by: Mohammad Ayaan

Lead Auditors:

- Mohammad Ayaan

Table of Contents

Table of Contents
Protocol Summary
Disclaimer
Risk Classification
Audit Details
  Scope
  Roles
Executive Summary
  Issues found
Findings
High
  [H-1] Password stored on-chain without encryption, ANYONE can read it
  [H-2] `setPassword` has no access controls, means a non-owner can change the password
Medium
Low
Informational
  [I-1] Compiler version is outdated
  [I-2] Incorrect natspec in `getPassword`
Gas

Protocol Summary

PasswordStore is a protocol that lets a single owner store a private password on-chain and retrieve it later. Only the owner should be able to set the password (`setPassword`) and read it (`getPassword`).

Disclaimer

Mohammad Ayaan makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond to the following commit hash:

```
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Scope

```
./src/
└── PasswordStore.sol
```

Roles

- **Owner:** The only person who should be able to set and read the password.
- **Outsiders:** No one else should be able to set or read the password.

Executive Summary

Add some notes about how the audit went, types of things you found, things you learned, etc.

Issues found

Severity	Number of issues found
High	2
Medium	0
Low	0
Informational	2
Gas	0
Total	**4**

Findings

High

[H-1] Password stored on-chain without encryption, ANYONE can read it

Description: All data stored on-chain is visible to anyone and can be read directly from the blockchain. The `s_password` variable is intended to be a private variable and only accessed through the `getPassword` function, which is intended to be only called by the owner of the contract.

We show one such method of reading any data off chain below.

Impact: Anyone can read the private password, severely breaking the functionality of the protocol.

Proof of Concept: The below steps prove that anyone can easily read the private password from the blockchain.

1. Run the local anvil chain using the anvil command:

```
make anvil
```

2. Run the deployment command:

```
make deploy
```

3. Run the cast storage command to read slot 1, where `s_password` is stored:

```
cast storage <Address> 1 --rpc-url http://127.0.0.1:8545
```

You will get something like this:

```
0x6d7950617373776f726400000000000000000000000000000000000000000014
```

4. Run the cast parse-bytes32-string command on that returned value:

```
cast parse-bytes32-string <Bytes32Object>
```

You will get the password in plain text:

```
myPassword
```

Recommended Mitigation: Because of this, the entire architecture of the contract should be rethought. We usually send example code to help with the fix, but in this case the architecture itself is flawed, so the team needs to think through a better design from scratch.

One possible approach is to encrypt the password off-chain and store only the encrypted value on-chain. This would require the user to remember a second password off-chain in order to decrypt the original. You would also likely want to remove the view function, so the password can never be exposed just by calling the contract.

[H-2] `setPassword` has no access controls, means a non-owner can change the password

Description: The `setPassword` function has no access control and is `external`, allowing anyone to set the password. However, the natspec comment indicates that the purpose is for only the `owner` to set the password.

```
    /*
     * @notice This function allows only the owner to set a new password.
     * @param newPassword The new password to set.
     */
    // @AUDIT-HIGH:: This function has no access control, allows anyone to set the
password, not just the owner. This is a critical security flaw!
    function setPassword(string memory newPassword) external {
        s_password = newPassword;
        emit SetNetPassword();
    }
```

Impact: Anyone can set/change the password of the contract, severely breaking the contract's intended functionality.

Proof of Concept: Add the `test_anyone_can_set_password_AUDIT` test function inside the `PasswordStore.t.sol` file. Run the test using `forge test --mt test_anyone_can_set_password_AUDIT`. It will pass, proving that `setPassword` can be called by a random address that is not the owner.

▼ **Here's the full code**

```
    // @AUDIT-HIGH:: This test demonstrates that anyone can set the password, not
just the owner. This is a critical security flaw!
    function test_anyone_can_set_password_AUDIT(address randomAddress) public {
        vm.assume(randomAddress != owner);
        vm.prank(randomAddress);
        string memory newPassword = "hackedPassword";
        passwordStore.setPassword(newPassword);
        vm.prank(owner);
        string memory actualPassword = passwordStore.getPassword();
```

```
        assertEq(actualPassword, newPassword);
    }
```

Recommended Mitigation: Add an access control conditional to the `setPassword` function. You could also use an `onlyOwner` modifier.

```
if (msg.sender != s_owner) {
    revert PasswordStore__NotOwner();
}
```

Medium

No medium severity findings.

Low

No low severity findings.

Informational

[I-1] Compiler version is outdated

Description: The compiler version is old. Since this is a new protocol, a newer compiler version is expected.

Impact: No impact on the codebase.

Proof of Concept: The contract uses an old Solidity version:

```
pragma solidity 0.8.18;
```

Recommended Mitigation: Use a more recent Solidity compiler version:

```
pragma solidity ^0.8.34;
```

[I-2] Incorrect natspec in `getPassword`

Description: The natspec comment in `getPassword` indicates that there is a parameter `newPassword`, but the function takes no parameters.

```
    /*
     * @notice This allows only the owner to retrieve the password.
     * @param newPassword The new password to set.
     */
    // @AUDIT-Minor:: Wrong documentation. It says "param newPassword" but the
function does not take any parameters. This is a minor issue but should be fixed for
clarity.
    function getPassword() external view returns (string memory) {
        if (msg.sender != s_owner) {
            revert PasswordStore__NotOwner();
        }
        return s_password;
    }
```

Impact: The incorrect documentation can confuse auditors and developers. If someone acts on it and adds a `newPassword` parameter, it will cause build and deployment errors.

Proof of Concept: Add the parameter `newPassword` to the smart contract function and all tests will break.

Recommended Mitigation: Remove the incorrect natspec line:

```
-    * @param newPassword The new password to set.
```

Gas

No gas findings.